Allow programmers to control coroutine elision

Document #:	P2477R3
Date:	2022-6-28
Project:	Programming Language C++
Audience:	Evolution Working Group

Reply-to:	Chuanqi Xu <chuanqi.xcq@alibaba-inc.com>

Contents

§ 1 Abstract

It is a well-known problem that coroutine needs dynamic allocation to work. Although there is a compiler optimization called coroutine elision which aims to eliminate dynamic allocations for coroutines, it doesn't work effectively and programmers couldn't control it in a well-defined way. So here we propose a method to allow programmers to control coroutine elision explicitly.

§ 2 Change Log

§ 2.1 R3

— Rewrite the proposal to make it easier to read.

— Propose to evaluate `promise_type::must_elide(P0, ..., Pn)` first. The design mimics the current design of coroutines to try construct `promise_type` from `promise_type(P0, ..., Pn)` and call `promise_type::new(std::size_t, P0, ..., Pn)` to allocate coroutine state at first.

— Add a case study for the asynchronous case to explain why the compiler couldn't handle it and why the programmers could understand it better than the compiler.

— Don't propose to detect coroutine elision due to the limited use case.

— Discuss the relationship between coroutine elision and the total memory usage.

— More examples.

2.2 R2

— Change the return type of `must_elide` from boolean to a tristate enum class.

— Add more backgrounds in the `Background and Motivation` section.

— Add a discussion talking about making `must_elide` a non-constexpr function.

2.3 R1

— Adding more examples. Many thanks for Lewis Baker offering these examples!

— Rename `should_elide` to `must_elide`.

— Discussing the semantics of the storage lifetime of elided coroutine frame.

— Adding more words about necessity, risks, and design ideas.

— Correcting that `coroutine_handle<>::elided()` and `coroutine_handle<PromiseType>::elided()` should be a runtime constant instead of a compile constant.

— Discussing how should we support `coroutine_handle<>::elided`.

— Raising the problem that indirect calls couldn't be elided.

— Discussing the relation of object and coroutine frame.

— Add a conclusion section.

3 Background and Motivation

A coroutine needs space to store information (coroutine status) when it suspends. Generally, a coroutine would use `promise_type::operator new` (if existing) or `::operator new` to allocate coroutine status. However, it is expensive to allocate memory dynamically. Even if we could use user-provided allocators, it is not easy (or very hard) to implement a safe and effective allocator. And dynamic allocation wouldn't be cheaper than static allocation after all.

To mitigate the expenses, Richard Smith and Gor Nishanov designed HALO (which is called `coroutine elision` nowadays) to allocate coroutine on the stack when the compiler finds it is safe to do so. There is a corresponding optimization called `CoroElide` in Clang/LLVM. But it is not mentioned in the C++ specification. The document only says:

```
[dcl.fct.def.coroutine]p9
An implementation **may** need to allocate additional storage for a
        coroutine.
```

So it leaves the space for the compiler to do optimization. But the programmers couldn't find a well-defined method to control coroutine elision. Also the `coroutine elision` is not guaranteed to happen just like other compiler optimizations. An elidable source code may become unelidable in a newer version of the same compiler. A recent example could be found at: https://github.com/llvm/llvm-project/issues/56262.

And `coroutine elision` doesn't like other compiler optimization which is transparent to programmers. Programmers have a strong feeling about dynamic allocation. An example could be found at: https://stackoverflow.com/questions/71581531/halo-support-on-recent-compilers-for-c-coroutines/72770957.

And the coroutine elision optimization in the compiler side is not satisfying, too. Although it works fine for simple cases, it couldn't optimize complex(asynchronous) cases. And it is hopeless to solve it in the compiler side in the recent future. However, the programmers are capable of analyzing the safety here. So the programmers pay for what they could have avoided. I think it is a violation of the zero abstract rule.

Except for the general need to avoid dynamic allocation to speed up. I heard from Niall Douglas that it is needed to disable `coroutine elision` in an embedded system. In the resource-limited system, we could use `promise_type::operator new` to allocate space from a big static-allocated bytes array. In this case, it would be annoying if the programmer found that his coroutine get elided surprisingly. And the programmer has nothing to do to disable it.

For the above reasons, we think it is necessary to provide a mechanism to control the coroutine elision in the specification.

4 Proposed Solutions

4.1 enum class coro_elision

To describe the requirement for eliding a coroutine. We need to introduce an enum class `coro_elision` to the standard library. The definition of `coro_elision` should be:

```
enum class coro_elision {
    may,
    always,
    never
};
```

§ 4.2 Terminology

§ 4.2.1 Coroutine Call

Let's say a coroutine call is a direct call to a coroutine function with a reachable definition.

§ 4.3 Informal Description for coroutine elision

The coroutine elision is only meaningful for the coroutine calls. When the coroutine elision is performed on a coroutine call, the necessary coroutine state for that coroutine call would be a local variable in the enclosing block.

(This implies that we could skip to allocate and deallocate the additional storage for the coroutine.)

§ 4.4 Informal Description for controlling coroutine elision

For every coroutine call `C(P0, ..., Pn)` with promise type `promise_type`, the compiler would try to evaluate `promise_type::must_elide(P0, ..., Pn)` in the constexpr time. If the evaluated result is `std::coro_elision::always`, the coroutine elision for this call is guaranteed to perform. And if the evaluated result is `std::coro_elision::never`, the coroutine elision for this call is guaranteed to not perform.

Otherwise (the result is `std::coro_elision::may` or `promise_type::must_elide(P0, ..., Pn)` is not a valid constexpr expression), the compiler would try to evaluate `promise_type::must_elide()` in the constexpr time. If the evaluated result is `std::coro_elision::always` or `std::coro_elision::never`, the coroutine elision for this call is guaranteed to perform or not perform according to the corresponding result. Otherwise, the behavior is implementation-defined.

5 Examples

A set of examples could be found at: https://github.com/ChuanqiXu9/llvm-project/tree/P2477/P2477_Examples

Here we share a usage to control the coroutine elision at the callsite by the default argument.

5.1 Controlling coroutine elision by the default argument

The following example implements a strategy to allow us to control the coroutine elision at the callsite by the default argument.

```
struct TaskPromiseAlternative : public TaskPromiseBase {
    // Return the last argument if its type is std::coro_elision.
    // Return std::coro_elision::may otherwise.
    template <typename... Args>
    static constexpr std::coro_elision must_elide(Args&&... args) {
        using ArgsTuple = std::tuple<Args...>;
                            constexpr    std::size_t    ArgsSize    =
        std::tuple_size_v<ArgsTuple>;
        if constexpr (ArgsSize > 0) {
              using LastArgType = std::tuple_element_t<ArgsSize - 1,
        ArgsTuple>;
                    if  constexpr  (std::is_convertible_v<LastArgType,
        std::coro_elision>)
                                      return  std::get<ArgsSize  -  1>
        (std::forward_as_tuple(args...));
        }

        return std::coro_elision::may;
    }
};

using ControllableTask = Task<TaskPromiseAlternative>;
```

The usage looks like:

```
ControllableTask    an_elide_controllable_coroutine(std::coro_elision
        elision_state = std::coro_elision::may) {
    co_return 43;
}

NormalTask example(int count) {
  int sum = 0;
  for (int i = 0; i < count; ++i) {
                                          auto          t          =
        an_elide_controllable_coroutine(std::coro_elision::always);
```

```
    sum += co_await std::move(t);
  }

  co_return sum;
}

NormalTask example2(int count) {
  std::vector<TaskBase *> tasks;
  for (int i = 0; i < count; ++i)
    // Or
    //    tasks.push_back(an_elide_controllable_coroutine());
    tasks.push_back(an_elide_controllable_coroutine(std::coro_elision::never));

  co_await whenAll(tasks);
  // ...
}
```

For example1, the lifetime of the coroutines of `an_elide_controllable_coroutine` is limited to this loop, so it is safe to elide them. But it is not true for example2, since we're going to resume the coroutines after the loop ended.

6 A case study for asynchronous coroutines

In this section, we would discuss 2 things:

1. Why the compiler can't handle complex (asynchronous in this context) situations
2. Why the programmers are capable of analyzing it.

The source code for the case study could be found at: https://github.com/ChuanqiXu9/llvm-project/tree/P2477/P2477_Examples/AsyncCase

The example implements a simple executor based on a simple thread pool and a simple task. When we `co_await` the simple task, the job to resume the awaited task is submitted to the simple executor.

```
template <typename U>
void await_suspend(std::coroutine_handle<U> continuation) noexcept {
    handle.promise().continuation = continuation;
    std::function<void()> func = [h = handle]() mutable {
        h.resume();
    };
    assert(handle.promise()._executor);
```

```
    handle.promise()._executor->schedule(func);
}
```

Then the executor would push the job to a random thread job queue and then the awaited task is waiting to be resumed. Once the awaited task is completed, the awaited task would resume the suspended task in the final suspend point:

```
struct FinalAwaiter {
    bool await_ready() const noexcept { return false; }
    template <typename PromiseType>
          auto   await_suspend(std::coroutine_handle<PromiseType>   h)
        noexcept {
        return h.promise().continuation;
    }
    void await_resume() noexcept {}
};
FinalAwaiter final_suspend() noexcept { return {}; }
```

When the suspended task is resumed by the awaited task, the suspended task is going to destroy the awaited task:

```
auto await_resume() noexcept {
    auto value =  handle.promise()._value;
    handle.destroy();
    return value;
}
```

6.1 Compiler Side

(The readers could skip this part if not interested)

When the compiler wants to perform the coroutine elision for a coroutine call, the most important thing it ensures if it is safe to do so.

The condition to ensure the safety is:

> Every path from the creation point of the coroutine to the exit point of the enclosing scope would meet the destroy point of the coroutine.

So when we `co_await` a asynchronous task,

```
auto task = task_creation();
co_await task;
```

the generated code would look like:

```
task = task_creation();
task.promise().continuation = current_task;
std::function<void()> func = [h = task]() mutable {
```

```
    h.resume();
};
task.promise()._executor->schedule(func);
<suspend-points>
task.destroy();
```

Although it **looks like** the creation point is covered by the destroy point, it is not true. The compiler has no idea that `current_task` would be resumed after all unless the implementation of `schedule` is too easy, which wouldn't be true in real cases. So the `<suspend-points>` here is an exit point indeed.

Also, the compiler couldn't infer the information that the `current_task` wouldn't be resumed before the `task` completes. From the perspective of the compiler, it could only find the coroutine frame of the `current_task` is escaped and the compiler would only assume the `current_task` might be resumed or destroyed at any point.

So from the perspective of the compiler, it couldn't be sure if the `task` would be destroyed finally nor if the `current_task` would be destroyed before the `task`. So the compiler couldn't do anything.

Note that things may be more complicated for the compiler. e.g., the coroutine type doesn't contain std::coroutine_handle only or there are more operations.

6.2 Programmers Side

From the programmers' side, we could know the fact easily:

> The `current_task` wouldn't be resumed before `task` get completed.

Given this fact, we know it is safe to make the coroutine state of `task` to be a local variable of the enclosing block. Since the local variable wouldn't be destructed before the `task` gets completed.

We could know the fact if:

1. We're the original author of the framework.
2. We heard the information from the author.
3. This is formally documented.
4. …

After all, it is easy for the programmers to the lifetime model for a specific coroutine type and a specific scheduling framework from my experiences.

The point of the section is that the programmers could understand the higher-level semantics more than the compiler so that we could make better decisions.

7 Limitations

The limitation of the proposal is that it couldn't handle the indirect call or the function definition is unreachable. This is a necessary condition. Otherwise, the compiler couldn't know if the function is a coroutine or not. The limitation should be unsolvable from my perspective.

My idea is to make it an undefined behavior if the evaluated result of `must_elide` is `coro_elision::always` but the definition is not reachable and the call is indirect.

The reason for the decision is that the compiler is not capable to forbid these cases since the compiler wouldn't know if the called function is a coroutine or not. I know it is possible to forbid something and say "no diagnostic is required". But I am not sure if it is too strict.

(The indirect call contains virtual functions and function pointers. The unreachable function definition refers to the coroutine functions which is defined in other TU. But I feel the unreachable function definition is much more precise.)

8 Concerns

The main concern I received is the potential misuse of the feature.

An example may be:

```
Task<int> example2(int task_count) {
  std::vector<Task<int>> tasks;
  for (int i = 0; i < task_count; i++)
    // The coroutine state of coro_task() becomes a local
    // variable of the loop.
    tasks.emplace_back(coro_task(std::coro_elision::always));

  // The semantics of whenAll is that it would
  // complete after all the tasks are completed.
  //
  // The tasks would contain to dangling pointers only.
  co_return co_await whenAll(std::move(tasks));
}
```

For the above example, `tasks` would contain a series of `Task<int>`, which contains a pointer to the corresponding coroutine frame. But if we force the coroutine

elision, `tasks` would contain a series of dangling pointers. The behavior is undefined.

And my response is: this is not a silver bullet and it has cost. It requires the users to understand the lifetime model of that coroutine they are using. And if the users don't think they understand it, it is OK to not use it.

I think the key point here is whether or not we're going to bring coroutine elision for users. If the answer is yes, no matter what the solution is, we would meet the same problem after all. This is the price we need to pay.

But I don't feel the price is too high. On the one hand, the lifetime model is not so hard to understand. On the other hand, it is OK to not use it.

A similar example I am wondering about is the memory order and consistency. I know many C++ programmers don't understand it clearly and it is much more complex than the lifetime model for specific coroutines.

§9 Wording

No formal wording yet. Let's go for it if we could get some consensus.

§10 Implementation

An implementation for clang could be found at: https://github.com/ChuanqiXu9/llvm-project/tree/P2477

If anyone wants a try, remember to add the `O1/2/3` option since this is not implemented in O0.

For other compilers, I think it might not be too hard to implement this. Since the hardest part to implement coroutine elision is the analysis part. But the proposal offloads the analysis burden to the users. So I think it is implementable.

§11 Summary

The paper proposes to introduce coroutine elision to the standard and provide a mechanism for the users to control coroutine elision. In this way, the users are capable to avoid some unnecessary dynamic allocations. The method wouldn't bring any breaking change by design and it is relatively easy to implement. The main limitation is that it couldn't handle indirect calls and unreachable function definitions. The main concern is the potential UB if the users don't use it properly.

Both the limitation and the concern are unavoidable as long as we want to introduce coroutine elision to the standard.

12 Q&A

12.1 What's the relationship between Coroutine elision and Copy Elision?

There is no relationship between copy elision and coroutine elision. But it might be confusing indeed. Here is an example:

```
template<typename... Args>
Task<void> foo_impl(Args... args) { co_await whatever(); ... }

template<typename... Args>
Task<void> foo(std::tuple<Args> t) {
  co_await std::apply([](Args&... args) { return foo_impl(args...);
        }, t);
}
```

Here the return object of `foo_impl` is guaranteed to be copy elided. But it doesn't affect coroutine elision. When coroutine elision performs for the call to `foo_impl()`, the coroutine state becomes a local variable of the enclosing block, which would be destructed later. So we shouldn't force coroutine elision on `foo_impl()`.

12.2 Is it good to connect coroutine elision and copy elision?

Probably not. The reason why people would be confused by the above example at the first sight is that the implementation of `Task` would combine it with the coroutine frame tightly. But this is semantic of `Task`, a specific coroutine type. It is not semantic of coroutine. We have no guarantee between the return object of the coroutine and the coroutine state at the language level.

12.3 Would the compiler now perform coroutine elision stably for synchronous cases?

No. It still depends on the complexity of the code pattern.

12.4 What will happen if we perform coroutine elision inside another coroutine?

```
Task foo(std::coro_elision kind) {
    co_return 43;
}
Task bar() {
    co_return co_await foo(std::coro_elision::always);
}
```

For the above example, the coroutine state of the call to `foo()` would become a local variable of `bar()`. Since `bar()` is a coroutine too, it has its coroutine state. Depends on the result of compiler optimization, the local variable may live in the coroutine state of `bar()`. Assume the variable lives in the coroutine state of `bar()`. The coroutine state of `bar()` might look like:

```
struct coro_state_of_bar {
  void (*resume_func)(coro_state_of_bar*);
  void (*destroy_func)(coro_state_of_bar*);
  promise_type;
  struct coro_state_of_foo {
      void (*resume_func)(coro_state_of_foo*);
      void (*destroy_func)(coro_state_of_foo*);
      promise_type;
      // Other information needed.
  };
  coro_state_of_foo state_of_foo;
  // Other information needed.
};
```

Given a call to bar() is not elided, the required size for the allocation would be `SizeOfOriginalBarFrame + SizeOfFooFrame`. So in this case, the times of allocating get decreased but the required size of one allocation goes higher. And the peak used memory keep the same with the unelided case.

§ 12.5 Is it possible that the total used memory becomes lower after we elide more coroutines

This is a trivial case. Presented for symmetry.

§ 12.6 Is it possible that the total used memory becomes higher after we elide more coroutines

This is possible too. See the following example:

```
Task foo(std::coro_elision kind) {
    co_return 43;
}
Task bar() {
    auto r1 = co_await foo(std::coro_elision::always);
    auto r2 = co_await foo(std::coro_elision::always);
```

```
    co_return (r1 + r2);
}
```

The peak used memory size would be `SizeOfOriginalBarFrame + 2 * SizeOfOriginalFooFrame`. But for the never elide case:

```
Task foo(std::coro_elision kind) {
    co_return 43;
}
Task bar() {
    auto r1 = co_await foo(std::coro_elision::never);
    auto r2 = co_await foo(std::coro_elision::never);
    co_return (r1 + r2);
}
```

The peak used memory size would be `SizeOfOriginalBarFrame + SizeOfOriginalFooFrame` since the second call to `foo()` wouldn't happen if the first frame is not destroyed.

Note1: The readers who are familiar with allocators may want to argue that the allocator wouldn't clean the memory immediately when `free()` is called. So the above example is not accurate. Yeah, it is true. So we might not be able to observe this when the scale is too small.

Note2: Maybe readers want to ask if the compiler can optimize the above example to make the frame of foo share one position. For the above case, I think it is possible ideally. But there will always be cases the compiler couldn't cover.

12.7 Would the allocation counts go higher if we elide more coroutines

It shouldn't be true.

13 Acknowledgments

Many thanks to Lewis Baker, Mathias Stearn, and Gašper Ažman for reviewing the series of papers.